MATH 128a: Homework #9

Instructor: F. Weber

Due by: **Wednesday, November 5, 2025, 11:59pm**

You may use any Matlab functions you find on the bcourses website to solve these problems. If you do, make sure to mention it.

Problem 1: Problems from the textbook, chapter 3.2

Solve problems 6, 7 of Section 3.2 of the textbook.

(If the problem has a different number in the 9th edition book, it is in parentheses)

Answer of Problem 1

Exercise 3.2.6. Solution.

$$x_0 = 0, \quad x_1 = 0.4, \quad x_2 = 0.7, \quad x = 0.5,$$
$$P_0 = f(x_0) = 0, \qquad P_1 = f(x_1) = 2.8, \qquad P_{0,1}(x) = 3.5, \qquad P_{0,1,2}(x) = \frac{27}{7}.$$

Neville's recurrence is

$$P_{i,j}(x) = \frac{(x - x_i)P_{i+1,j}(x) - (x - x_j)P_{i,j-1}(x)}{x_j - x_i}.$$

First use it for $P_{1,2}(x)$ (which involves the unknown $P_2 = f(x_2)$):

$$P_{1,2}(x) = \frac{(x - x_1)P_2 - (x - x_2)P_1}{x_2 - x_1} = \frac{(0.5 - 0.4)P_2 - (0.5 - 0.7) \cdot 2.8}{0.7 - 0.4} = \frac{0.1P_2 + 0.56}{0.3}.$$

Next use it for $P_{0,1,2}(x)$ with the known $P_{0,1}(x) = 3.5$:

$$P_{0,1,2}(x) = \frac{(x - x_0)P_{1,2}(x) - (x - x_2)P_{0,1}(x)}{x_2 - x_0} = \frac{0.5\,P_{1,2}(x) + 0.2 \cdot 3.5}{0.7}.$$

Since $P_{0,1,2}(x) = \dfrac{27}{7}$, we get

$$\frac{27}{7} = \frac{0.5\,P_{1,2}(x) + 0.7}{0.7} \quad \Longrightarrow \quad 2.7 = 0.5\,P_{1,2}(x) + 0.7 \quad \Longrightarrow \quad P_{1,2}(x) = 4.$$

Solve for P_2 from the first relation:

$$4 = \frac{0.1P_2 + 0.56}{0.3} \quad \Longrightarrow \quad 0.1P_2 + 0.56 = 1.2 \quad \Longrightarrow \quad P_2 = 6.4.$$

$$\boxed{P_2 = f(0.7) = 6.4}$$

Exercise 3.2.7. Solution.

$$\text{Given: } x_j = j,\ j = 0, 1, 2, 3.$$
$$P_{0,1}(x) = 2x + 1, \quad P_{0,2}(x) = x + 1, \quad P_{1,2,3}(2.5) = 3.$$

We want to find $P_{0,1,2,3}(2.5)$.

Neville's formula:

$$P_{i,j}(x) = \frac{(x - x_i)P_{i+1,j}(x) - (x - x_j)P_{i,j-1}(x)}{x_j - x_i}.$$

Apply this for $P_{0,1,2,3}(x)$:

$$P_{0,1,2,3}(x) = \frac{(x - x_0)P_{1,2,3}(x) - (x - x_3)P_{0,1,2}(x)}{x_3 - x_0}.$$

At $x = 2.5$:

$$P_{0,1,2,3}(2.5) = \frac{(2.5 - 0)P_{1,2,3}(2.5) - (2.5 - 3)P_{0,1,2}(2.5)}{3 - 0} = \frac{2.5(3) + 0.5\,P_{0,1,2}(2.5)}{3}.$$

So we need $P_{0,1,2}(2.5)$.

Use Neville's formula again:

$$P_{0,1,2}(x) = \frac{(x - x_0)P_{1,2}(x) - (x - x_2)P_{0,1}(x)}{x_2 - x_0}.$$

But we don't have $P_{1,2}(x)$ yet. We can find it from $P_{0,1}(x)$ and $P_{0,2}(x)$:

$$P_{0,1,2}(x) = \frac{(x - x_0)P_{1,2}(x) - (x - x_2)P_{0,1}(x)}{x_2 - x_0} \implies P_{1,2}(x) = \frac{(x_2 - x_0)P_{0,1,2}(x) + (x - x_2)P_{0,1}(x)}{x - x_0}.$$

However, an easier way is to note that $P_{0,1}(x) = 2x + 1$ and $P_{0,2}(x) = x + 1$ correspond to interpolations through x_0, x_1, x_2. Hence at $x = 2.5$:

$$P_{0,1,2}(2.5) = \frac{(2.5 - 0)P_{1,2}(2.5) - (2.5 - 2)P_{0,1}(2.5)}{2 - 0}.$$

To find $P_{1,2}(2.5)$, use $P_{0,1}$ and $P_{0,2}$ at $x = 1, 2$:

$$P_{0,1}(1) = 2(1) + 1 = 3, \quad P_{0,2}(2) = 2 + 1 = 3.$$

This content is protected and may not be shared, uploaded, or distributed.

So $f(1) = 3, f(2) = 3$, meaning $P_{1,2}(x)$ is constant 3. Then

$$P_{0,1,2}(2.5) = \frac{(2.5-0)(3)-(2.5-2)(2(2.5)+1)}{2} = \frac{7.5-0.5(6)}{2} = \frac{7.5-3}{2} = 2.25.$$

Finally,

$$P_{0,1,2,3}(2.5) = \frac{2.5(3)+0.5(2.25)}{3} = \frac{7.5+1.125}{3} = \frac{8.625}{3} = 2.875.$$

$$\boxed{P_{0,1,2,3}(2.5) = 2.875.}$$

This content is protected and may not be shared, uploaded, or distributed.

Problem 2: Divided Differences Matlab

(a) Implement Newton's divided differences formula (Algorithm 3.2 in the textbook) in Matlab.

(b) Implement a matlab function that evaluates Newton's polynomial at a given point x given interpolation points $x_0, \ldots, x_n$ and the divided differences from (a).

Answer of Problem 2

```
function [c, D] = newton_divdif(x, y)
    % Basic checks
    if numel(x) ~= numel(y)
        error('x and y must have the same length.');
    end
    x = x(:);  % column vectors
    y = y(:);
    n = numel(x);

    % Disallow duplicate nodes
    if any(diff(sort(x)) == 0)
        error('Interpolation nodes in x must be distinct.');
    end

    % Build divided-difference table
    D = zeros(n, n);
    D(:,1) = y;                         % first column is f(x_i)
    for j = 2:n                         % column
        for i = 1:(n - j + 1)           % row
            D(i,j) = (D(i+1,j-1) - D(i,j-1)) / (x(i+j-1) - x(i));
        end
    end

    % Newton coefficients are the first row of D
    c = D(1, :).';
end

function p = newton_eval(xnodes, c, x)
    if numel(xnodes) ~= numel(c)
        error('xnodes and c must have the same length.');
    end
    xnodes = xnodes(:);
    c = c(:);

    p = c(end) * ones(size(x));           % start from highest-order term
    for k = numel(c)-1 : -1 : 1
        p = c(k) + (x - xnodes(k)) .* p;
    end
end
```

This content is protected and may not be shared, uploaded, or distributed.

Problem 3: Divided differences

Prove that $f[x_1, \ldots, x_k, x_0] = f[x_0, x_1, \ldots, x_k]$ for any $k \geq 1$.

Answer of Problem 3

Assume first that the nodes are pairwise distinct. Let $p \in \Pi_k$ be the (unique) polynomial of degree $\leq k$ that interpolates f at $x_0, \ldots, x_k$. By the definition of divided differences, the coefficient of x^k in p is precisely $f[x_0, \ldots, x_k]$.
Write p in its Lagrange form:

$$p(x) = \sum_{i=0}^{k} f(x_i)\, \ell_i(x), \qquad \ell_i(x) = \prod_{\substack{m=0 \\ m \neq i}}^{k} \frac{x - x_m}{x_i - x_m}.$$

The coefficient of x^k in $\ell_i(x)$ is $1/\prod_{m \neq i}(x_i - x_m)$, hence

$$f[x_0, \ldots, x_k] = \sum_{i=0}^{k} \frac{f(x_i)}{\prod_{m \neq i}(x_i - x_m)}. \tag{$*$}$$

The right-hand side is symmetric under any permutation of the nodes $\{x_0, \ldots, x_k\}$ (it merely relabels the summation index). Therefore $f[x_{\sigma(0)}, \ldots, x_{\sigma(k)}] = f[x_0, \ldots, x_k]$ for every permutation σ, and in particular $f[x_1, \ldots, x_k, x_0] = f[x_0, x_1, \ldots, x_k]$.
If some nodes coincide, define the divided differences by continuity (or via the Hermite formulas involving derivatives). Since the expression $(*)$ is continuous in the nodes, the same symmetry follows by a limiting argument.

This content is protected and may not be shared, uploaded, or distributed.

Problem 4: Problems from the textbook, chapter 3.3

Solve problems 2, 4, 7, 14(16), 19(10), 22(20) of Section 3.3 of the textbook.

(If the problem has a different number in the 9th edition book, it is in parentheses)

Answer of Problem 4

Exercise 3.3.2. Solution.

```
%% (a)  f(0.43) from data at 0, 0.25, 0.5, 0.75
x = [0 0.25 0.5 0.75];
y = [1 1.64872 2.71828 4.48169];
[c, ~] = newton_divdif(x, y);
xq = 0.43;

P1_a = newton_eval(x(1:2), c(1:2), xq);   % degree 1
P2_a = newton_eval(x(1:3), c(1:3), xq);   % degree 2
P3_a = newton_eval(x,       c,      xq);  % degree 3
% P1_a = 2.1158
% P2_a = 2.3764
% P3_a = 2.3606
```

```
%% (b)  f(0) from data at -0.5, -0.25, 0.25, 0.5
x = [-0.5 -0.25 0.25 0.5];
y = [1.93750 1.33203 0.800781 0.687500];
[c, ~] = newton_divdif(x, y);
xq = 0;

P1_b = newton_eval(x(1:2), c(1:2), xq);   % degree 1
P2_b = newton_eval(x(1:3), c(1:3), xq);   % degree 2
P3_b = newton_eval(x,       c,      xq);  % degree 3
% P1_b = 0.7266
% P2_b = 0.9531
% P3_b = 0.9844
```

Exercise 3.3.4. Solution.
For equally–spaced nodes $x_i = x_0 + ih$ with forward differences

$$\Delta f_i = f_{i+1} - f_i, \qquad \Delta^{k+1} f_i = \Delta^k f_{i+1} - \Delta^k f_i,$$

the degree–n Newton forward polynomial is

$$P_n(x) = f_0 + t\,\Delta f_0 + \frac{t(t-1)}{2!}\,\Delta^2 f_0 + \frac{t(t-1)(t-2)}{3!}\,\Delta^3 f_0 + \cdots, \qquad t = \frac{x - x_0}{h}.$$

This content is protected and may not be shared, uploaded, or distributed.

(a) Data at $x = 0, 0.25, 0.5, 0.75$:

$$f_0 = 1, \quad f_1 = 1.64872, \quad f_2 = 2.71828, \quad f_3 = 4.48169.$$

Forward differences:

$$\begin{aligned}\Delta f_0 &= f_1 - f_0 = 0.64872,\\ \Delta^2 f_0 &= (f_2 - f_1) - (f_1 - f_0) = 0.42084,\\ \Delta^3 f_0 &= \Delta^2 f_1 - \Delta^2 f_0 = 0.27301.\end{aligned}$$

With $h = 0.25$ and $x = 0.43$, we have $t = \dfrac{0.43 - 0}{0.25} = 1.72$. Thus

$$P_1(0.43) = f_0 + t\,\Delta f_0 = 1 + 1.72(0.64872) = \boxed{2.115798400},$$

$$P_2(0.43) = P_1(0.43) + \frac{t(t-1)}{2}\,\Delta^2 f_0 = 2.115798400 + \frac{1.72 \cdot 0.72}{2}(0.42084) = \boxed{2.376382528},$$

$$P_3(0.43) = P_2(0.43) + \frac{t(t-1)(t-2)}{6}\,\Delta^3 f_0 = 2.376382528 + \frac{1.72 \cdot 0.72 \cdot (-0.28)}{6}(0.27301) = \boxed{2.360604734}.$$

(b) Data at $x = 0.1, 0.2, 0.3, 0.4$:

$$\begin{aligned}&f_0 = -0.29004986, \quad f_1 = -0.56079734, \quad f_2 = -0.81401972, \quad f_3 = -1.0526302,\\ &\Delta f_0 = -0.27074748, \quad \Delta^2 f_0 = 0.01752510, \quad \Delta^3 f_0 = -0.00291320.\end{aligned}$$

With $h = 0.1$ and $x = 0.18$, we have $t = \dfrac{0.18 - 0.1}{0.1} = 0.8$. Therefore

$$P_1(0.18) = f_0 + t\,\Delta f_0 = -0.29004986 + 0.8(-0.27074748) = \boxed{-0.506647844},$$

$$P_2(0.18) = P_1(0.18) + \frac{t(t-1)}{2}\,\Delta^2 f_0 = -0.506647844 + \frac{0.8(-0.2)}{2}(0.01752510) = \boxed{-0.508049852},$$

$$P_3(0.18) = P_2(0.18) + \frac{t(t-1)(t-2)}{6}\,\Delta^3 f_0 = -0.508049852 + \frac{0.8(-0.2)(-1.2)}{6}(-0.00291320) = \boxed{-0.508143074}.$$

Exercise 3.3.7. Solution.
We use the Newton divided–difference form with nodes

$$x_0 = -0.1, \quad x_1 = 0, \quad x_2 = 0.2, \quad x_3 = 0.3 \qquad \text{(and } x_4 = 0.35 \text{ for part (b))}.$$

(a) Degree three. Given

$$f(x_0) = 5.30000, \quad f(x_1) = 2.00000, \quad f(x_2) = 3.19000, \quad f(x_3) = 1.00000,$$

the divided–difference table (top row of each column shown) is

x_i	$f[x_i]$	$f[x_i, x_{i+1}]$	$f[x_i, x_{i+1}, x_{i+2}]$	$f[x_i, \ldots, x_{i+3}]$
-0.1	5.3	-33	$\dfrac{779}{6}$	$-\dfrac{1670}{3}$

This content is protected and may not be shared, uploaded, or distributed.

so the Newton coefficients are

$$c_0 = 5.3, \qquad c_1 = -33, \qquad c_2 = \frac{779}{6}, \qquad c_3 = -\frac{1670}{3}.$$

Hence the degree–three interpolant is

$$\boxed{P_3(x) = 5.3 - 33\,(x+0.1) + \frac{779}{6}\,(x+0.1)\,x - \frac{1670}{3}\,(x+0.1)\,x\,(x-0.2)}.$$

(b) Degree four (add $x_4 = 0.35$, $f(x_4) = 0.97260$). Extending the table with the new point gives the additional divided difference

$$c_4 = f[x_0, x_1, x_2, x_3, x_4] = \frac{516016}{189} \approx 2{,}730.243386.$$

Therefore the degree–four interpolant is

$$\boxed{P_4(x) = P_3(x) + \frac{516016}{189}\,(x+0.1)\,x\,(x-0.2)\,(x-0.3)}.$$

(Any numerically rounded decimal versions of the coefficients are also acceptable: $c_2 \approx 129.833333$, $c_3 \approx -556.666667$, $c_4 \approx 2730.243386$.)

Exercise 3.3.14. Solution.
Since $P_3(x)$ is the Newton interpolating polynomial on $x_0 = 0$, $x_1 = 0.25$, $x_2 = 0.5$, $x_3 = 0.75$, we have $f(0.75) = P_3(0.75)$.

$$\begin{aligned} P_3(0.75) &= 1 + 4(0.75) + 4(0.75)(0.75 - 0.25) + \frac{16}{3}(0.75)(0.75 - 0.25)(0.75 - 0.5) \\ &= 1 + 3 + 4(0.75)(0.5) + \frac{16}{3}(0.75)(0.5)(0.25) \\ &= 1 + 3 + 1.5 + 0.5 = 6. \end{aligned}$$

$$\boxed{f(0.75) = 6.}$$

Exercise 3.3.19. Solution.
The nodes are equally spaced with step $h = 1$:

$$x = -2, -1, 0, 1, 2, 3, \qquad f(x) = 1, 4, 11, 16, 13, -4.$$

For equally spaced data, the interpolating polynomial has degree m iff the m-th forward differences are constant and the $(m+1)$-st ones vanish.
Compute forward differences (Δ):

x	-2	-1	0	1	2	3
f	1	4	11	16	13	-4
Δf	3	7	5	-3	-17	
$\Delta^2 f$	4	-2	-8	-14		
$\Delta^3 f$	-6	-6	-6			
$\Delta^4 f$	0	0				

This content is protected and may not be shared, uploaded, or distributed.

The third forward differences are constant ($\Delta^3 f \equiv -6 \neq 0$), while the fourth forward differences are identically zero. Hence the unique polynomial interpolating the data has degree three.

Exercise 3.3.22. Solution.
Let P_n be the degree-$\leq n$ Newton interpolating polynomial for f on $x_0, \ldots, x_n$; then $P_n(x_i) = f(x_i)$ for $i = 0, \ldots, n$. Define the function

$$R(t) := f(t) - P_n(t) - f[x_0, \ldots, x_n, x] \prod_{j=0}^{n} (t - x_j).$$

By construction,

$$R(x_i) = f(x_i) - P_n(x_i) - f[x_0, \ldots, x_n, x] \cdot 0 = 0 \quad (i = 0, \ldots, n),$$

and also

$$R(x) = f(x) - P_n(x) - f[x_0, \ldots, x_n, x] \prod_{j=0}^{n} (x - x_j) = 0,$$

(the last equality is the Newton form of the $(n+1)$-point interpolant). Hence R has $n+2$ distinct zeros: $x_0, \ldots, x_n, x$.
Applying Rolle's theorem $n+1$ times, there exists $\xi \in (\min\{x_0, \ldots, x_n, x\}, \max\{x_0, \ldots, x_n, x\})$ such that $R^{(n+1)}(\xi) = 0$. But

$$R^{(n+1)}(t) = f^{(n+1)}(t) - (n+1)!\, f[x_0, \ldots, x_n, x],$$

because P_n has degree $\leq n$ (so $P_n^{(n+1)} \equiv 0$) and

$$\frac{d^{n+1}}{dt^{n+1}} \prod_{j=0}^{n} (t - x_j) = (n+1)!.$$

Therefore $0 = R^{(n+1)}(\xi) = f^{(n+1)}(\xi) - (n+1)!\, f[x_0, \ldots, x_n, x]$, which yields

$$f[x_0, \ldots, x_n, x] = \frac{f^{(n+1)}(\xi)}{(n+1)!}.$$

This proves the assertion.

This content is protected and may not be shared, uploaded, or distributed.

Problem 5: Hermite Interpolation

Implement a function in matlab `[coeffs] = hermite(xi,fi,dfi)` that returns the coefficients of the Hermite polynomial using divided differences interpolating at values `(xi,fi,dfi)` (where `xi` are the values x_i `fi` are the values $f(x_i)$ and `dfi` are the values of the derivatives $f'(x_i)$).

Answer of Problem 5

```
function coeffs = hermite(x,y, d)
if ~isvector(x)
  error('hermite:input_error','x must be a vector of length > 2.')
end
% get size and orient
n = length(x);
x = x(:);
% make sure x is long enough, we can't construct an interpolating
% polynomial with just one point
if n < 2
  error('hermite:input_error','x must be a vector of length > 2.')
end
% check y and d
if isvector(y) && isvector(d) && length(y) == n && length(d) == n
  % orient
  y = y(:);
  d = d(:);
  m = 1;
elseif size(y,2) == n && size(d,2) == n && size(y,1) == size(d,1)
  m = size(y,1);
  y = y';
  d = d';
else
  error('hermite:input_error','y and d must be vectors or matrices of same size with length(x) columns.')
end
% sort breaks & data if needed
if ~issorted(x)
  [x x_ix] = sort(x);
  if m == 1
    y = y(x_ix);
    d = d(x_ix);
  else
    y = y(x_ix,:);
    d = d(x_ix,:);
  end
end
% compute coefficients
coef = zeros(m,n-1,4);
dx = diff(x);
for i = 1:m
  dy = diff(y(:,i));
  coef(i,:,4) = y(1:end-1,i)';
  coef(i,:,3) = d(1:end-1,i)';
  coef(i,:,2) = 3*dy./(dx.^2) - (2*d(1:end-1,i)+d(2:end,i))./dx;
  coef(i,:,1) = -2*dy./(dx.^3) + (d(1:end-1,i)+d(2:end,i))./(dx.^2);
end
% create the piecewise polynomial structure
pp = mkpp(x,coef,m);

coeffs = pp.coefs;
end
```

This content is protected and may not be shared, uploaded, or distributed.